

CMSC331 Programming Languages

Introduction

Slide sources:

- *Programming Languages 3/e,* Louden & Lambert, Cengage Learning



Topics for Today

- Syllabus, administrative stuff
- Overview of the semester
- A brief history of PLs
- Picking a PL



People

- Instructor: Prof. Richard Chang
- Teaching Assistants: Dharani Nadendla

You are …

- All computer science majors
- 58% sophomores, 35% juniors, 7% seniors
- 58% took CMSC 202 CS2 at UMBC in Spring 2026

  35% in Fall 2025, 3% in Spring 2025
- 79% are also taking CMSC 341 Data Structures

  10% in Spring 2026
- 24% are also taking CMSC 313 Comp Org. & Assembly

  17% took CSMC 313 previously

Taking 313, 331 and 341 (!!)

24 students attempted CMSC 313 + 331 + 341 Fall 2022

- 7/24 = 29% all A's
  - A rate for CMSC 341 was 22.5%
  - 6 were from my sections of 331
- 7/24 = 29% DFW in one of CMSC 313, 331 or 341
  - DFW rate for CMSC 341 was 18.4%
  - none from my sections of 331

Where to get info

- Class website: https://umbc.edu/~chang/cs331/
- Google Drive shared folder
- Google Calendar
- Mailing list archive: https://groups.google.com/g/cmsc331
- Email TA or Prof. Chang

Office Hours

- Prof. Chang:
  - Tuesday & Thursday 12:30pm – 1:30pm (ITE 326)
- TAs:
  - TBA

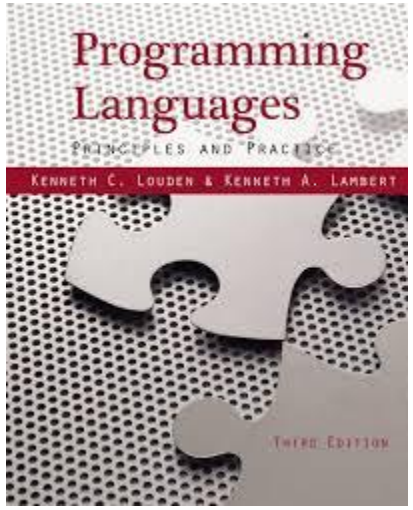
Check website & Google Calendar for exceptions

Additional Help …

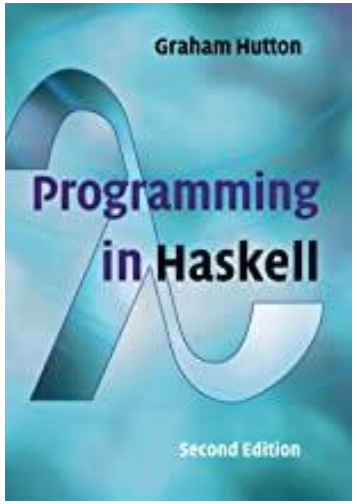
- Computing Success Center

  https://academicsuccess.umbc.edu/drop-in-tutoring/

Textbooks



Programming Languages: Principles and Practices, 3rd edition,
Kenneth C. Louden and Kenneth A. Lambert.
Cengage Learning, 2012 (ISBN: 9780124104099)



Programming in Haskell, 2nd edition,
Graham Hutton.
Cambridge University Press, 2016 (ISBN: 9781316626221)

Grading

Reading & Lecture Quizzes (12)	6%
Homework (10)	30%
Midterm Exams (2)	40%
Final Exam	24%



Homework Policy

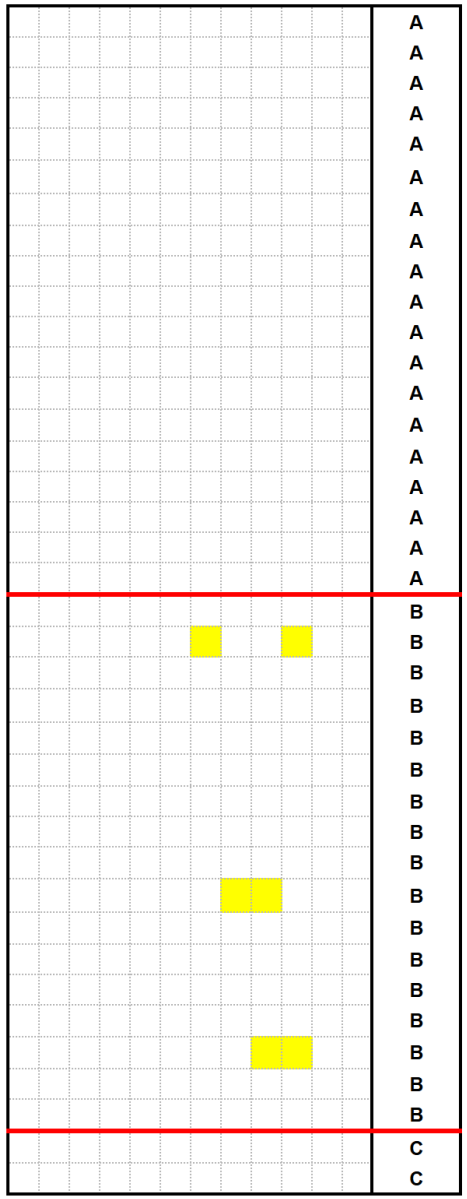
- HW due Thu 11:59pm
- Submit with Google Forms and/or on GL
- Allowed to discuss HW verbally
- No copying --- all cases of cheating will be reported
- **You** are responsible for **learning** from homework
- Up to 40% of grade programming assignments determined by **good usage** of Haskell features

Late Homework

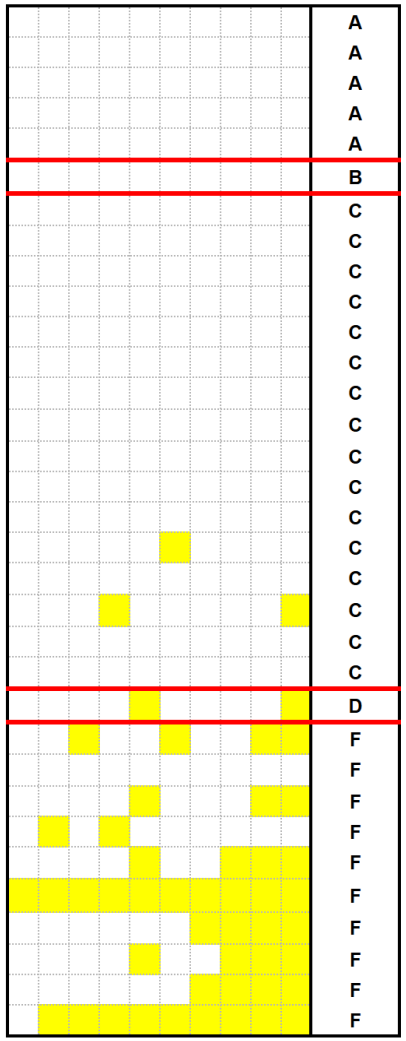
- Three late passes: due Thu, turned in Tue
- One late pass each for: HW1-4, HW5-7, HW8-10
- Late passes do not rollover or “stack”
- Unexcused late work:

1 day late (by Fri 11:59pm)	=	-5%
2 days late (by Sat 11:59pm)	=	-10%
3 days late (by Sun 11:59pm)	=	-20%
4 days late (by Mon 11:59pm)	=	-40%
last chance (by Tue 11:59pm)	=	-100%

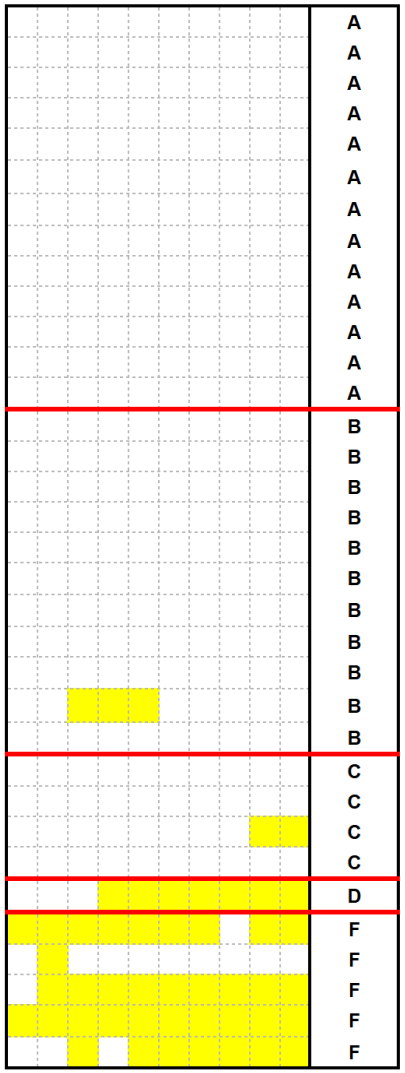
FALL 2024



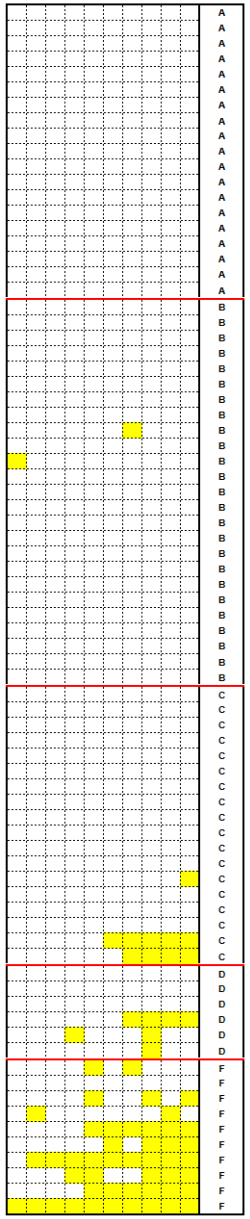
SPRING 2025



FALL 2025



SPRING 2026



Attendance in Lectures

- Fall 2024 attendance was required
- Use of electronics during lecture
- Slides are not a transcript of the lectures.

  If you miss lecture, you will miss out on:

  - …
  - …
  - …

AI Chats

- Review, alternate perspectives
- Deeper dive
- Fact check

More UMBC Title IX Policies
on the Office of Equity and Inclusion website…

Covers: Accessibility & disability accommodations; Sexual assault & harassment; Gender-based violence & discrimination; Pregnant & parenting students; religious observances & accommodations; Hate, bias, discrimination & harassment



Some perspectives on PL

- PL is a large field of computer science
- PL people are bossy
- Many ways to teach this class
  - "grand tour"
  - real languages vs "simplified" versions
  - compiler design?
  - proofs & semantic analysis?



A Brief History

of

Programming Languages

What you should be reading

Chapter 1

Introduction

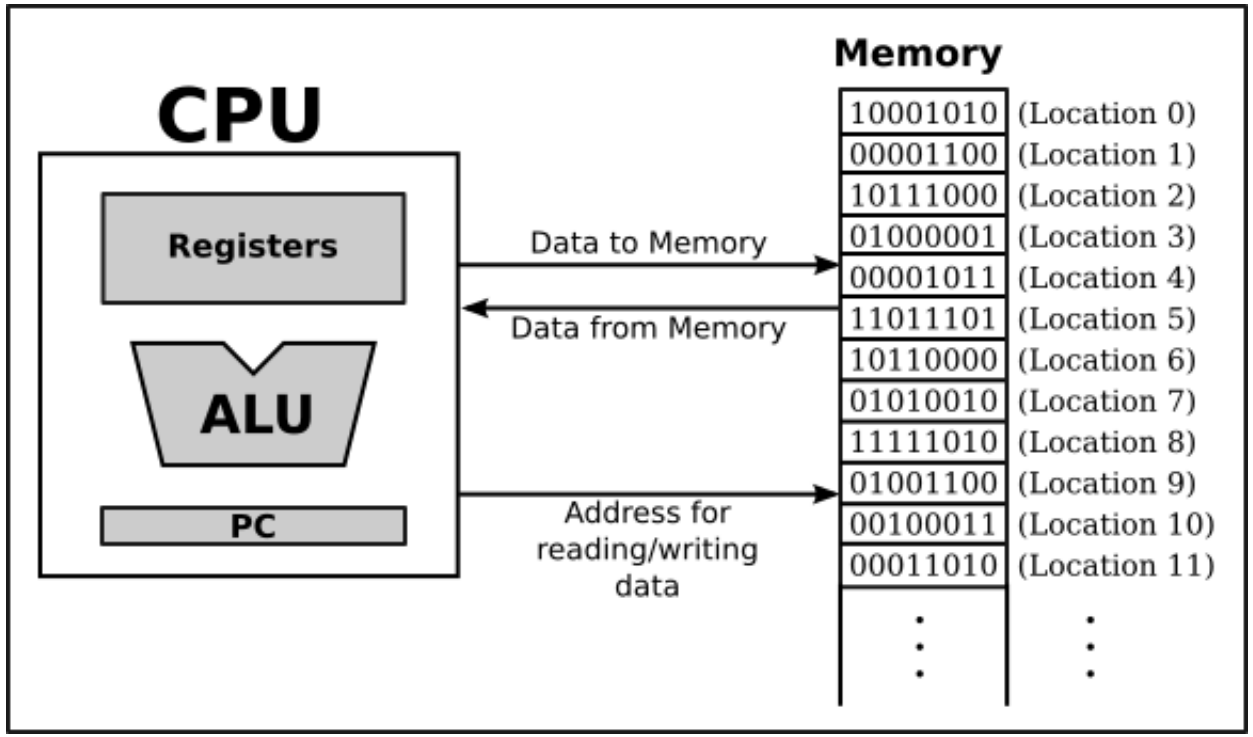
1.1 The Origins of Programming Languages 3
1.2 Abstractions in Programming Languages 8
1.3 Computational Paradigms 15
1.4 Language Definition 16
1.5 Language Translation 18
1.6 The Future of Programming Languages 19



Prehistory of programming

- **Devices that do one thing**
- **Configurable devices**
  - "hardwired"
  - OG FPGAs
- **First stored programs**
  - John von Neumann
  - fetch-execute cycle: reads and executes code in memory
  - binary code "toggled-in"
  - tedious and error-prone

Fetch-Execute Cycle



source: https://math.hws.edu/javanotes/c1/s1.html

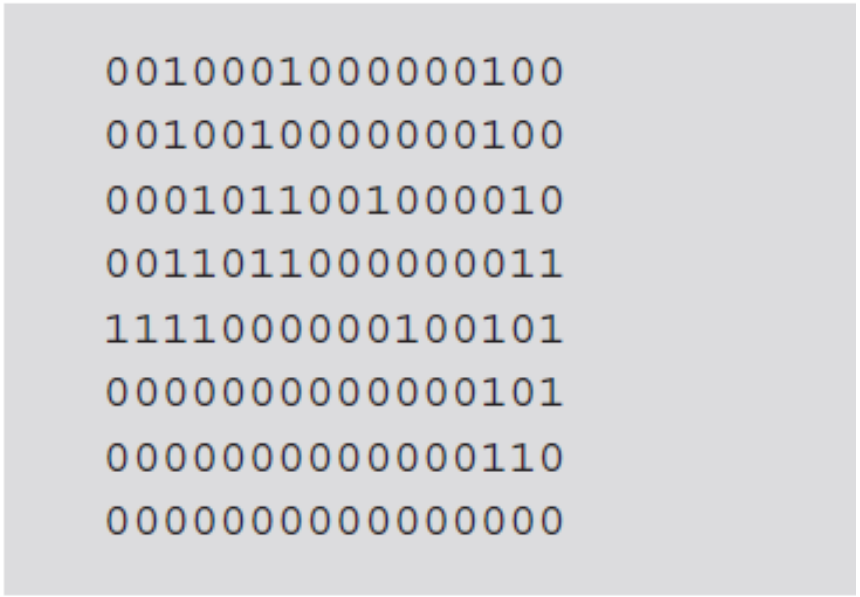


Figure 1.2 A machine language program

Assembly Language, Symbolic Codes, and Software Tools

- **Assembly language**: a set of mnemonic symbols for instruction codes and memory locations
  - Example: `LD R1, FIRST`
- **Assembler**: a program that translates the symbolic assembly language code to binary machine code
- **Loader**: a program that loads the machine code into computer memory
- **Input devices:**
  - Keypunch machine
  - Card reader



```
.ORIG x3000       ; Address (in hexadecimal) of the first instruction
LD  R1, FIRST     ; Copy the number in memory location FIRST to register R1
LD  R2, SECOND    ; Copy the number in memory location SECOND to register R2
ADD R3, R2, R1    ; Add the numbers in R1 and R2 and place the sum in
                  ; register R3
ST  R3, SUM       ; Copy the number in R3 to memory location SUM
HALT              ; Halt the program
FIRST  .FILL #5   ; Location FIRST contains decimal 5
SECOND .FILL #6   ; Location SECOND contains decimal 6
SUM    .BLKW #1   ; Location SUM (contains 0 by default)
.END              ; End of program
```

Figure 1.3 An assembly language program that adds two numbers

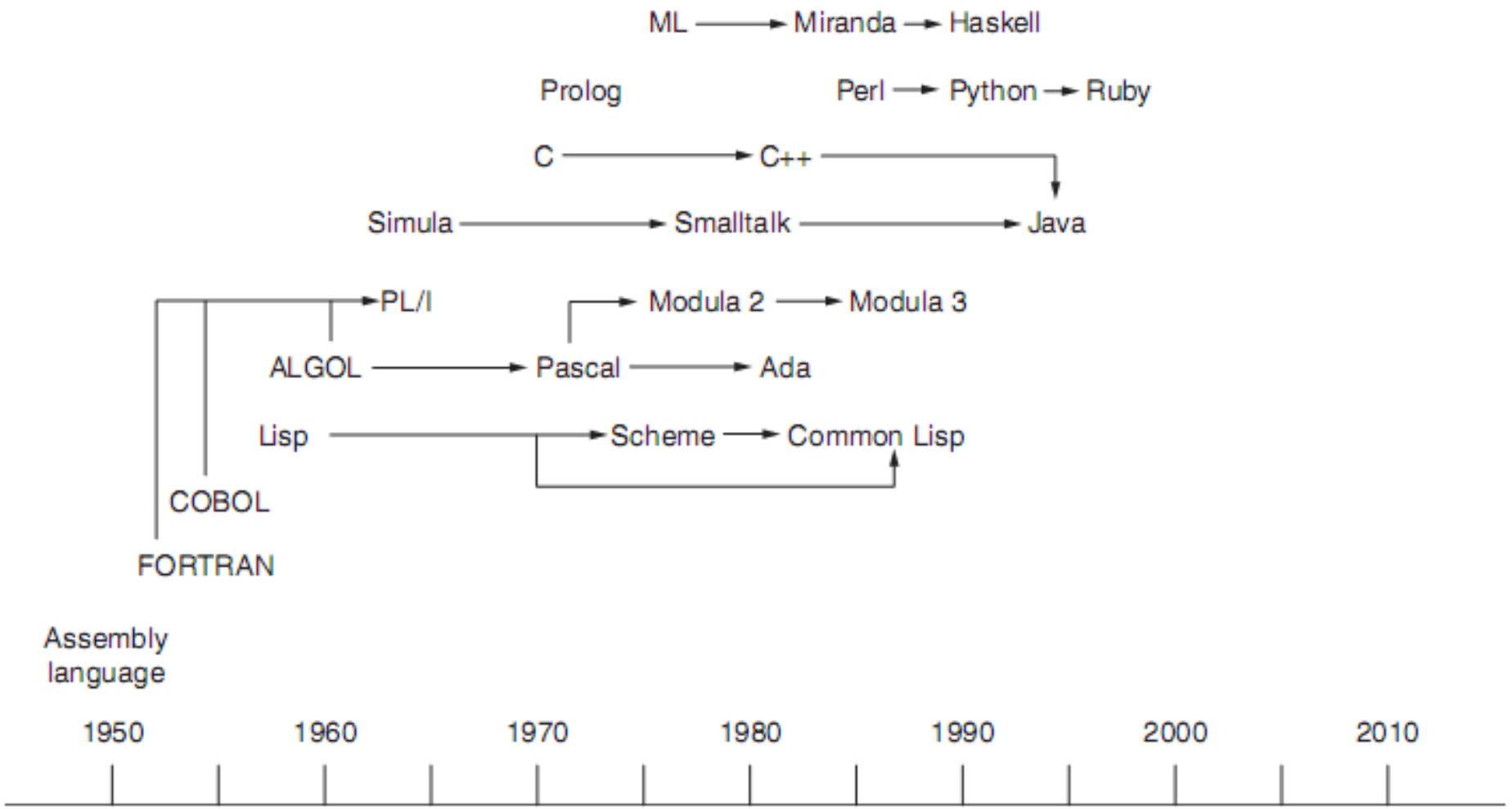


Figure 1.1 A programming language timeline



Four fuzzy categories of programming languages:

- Imperative (non-OOP): ALGOL, Basic, FORTRAN, Pascal, C
- Functional: LISP, Scheme, ML, Haskell
- Object-Oriented: Smalltalk, C++, Java, Python
- Logic: Prolog, (?)

Multi-paradigm PLs: OCaml, Scala, Ruby, Rust, F#



A Brief History of PLs

From: "History of Programming Languages," Wikipedia

1957 FORTRAN: first compiled language, IBM, scientific and engineering applications

1958 LISP: functional PL, AI, lists

1958 ALGOL: begin-end blocks, for loops, if statements ...

1959 COBOL: "English-like" PL, business applications



1964 PL/1: IBM's one PL to rule them all, first to use exceptions, pointers, ...

1967 Simula: first object-oriented PL

1970 Pascal: structured programming, college CS1

1970 FORTH: RPN, astronomers, easy to implement

1972 C: Unix, systems programming

1972 Smalltalk: "pure" object-oriented PL

1972 Prolog: logic programming language



1973 ML: functional, type inference, formally specified

1974 Scheme: functional, better LISP, MIT

1980 C++: OOP bolted onto C 🤮

1983 Ada: DoD's one PL to rule them all, big, awkward, required for all DoD software 1987-1997

1987 Perl: scripting PL, string processing, "making easy things easy and hard things possible"



1990 Haskell: "pure" functional PL, research

1991 Python: indentation counts, data science, CS1

1995 Java: OOP, "write once run anywhere", byte code, hyped security claims, Oracle/Sun's one PL to rule…

1997 Ruby: Multi-paradigm objected-oriented & functional …